

Analysis of Algorithms

Assignment 05

Name: Jose Ignacio Cabascango Fernandez

Link GitHub: <https://github.com/CodeWithJose1/Analysis-of-Algorithms/tree/main/Assignment-05/src>

1. Create a program that implements the Insertion Sort Algorithm. Your program has to be able to sort a randomly generated array of alphabetic characters. Decide if you have to do or not any adjustments for the algorithm to work with alphabetic characters instead of integer numbers.

It is not necessary to modify the insertion sort algorithm itself. The original algorithm can operate on integers; due to their natural ordering, alphabetic characters can be compared using their character codes. Therefore, the same comparison:

$$A[j] > v$$

can be used to determine their relative order. The main modification is the data type `int` is replaced by `char`. For example:

Before: H C A F B

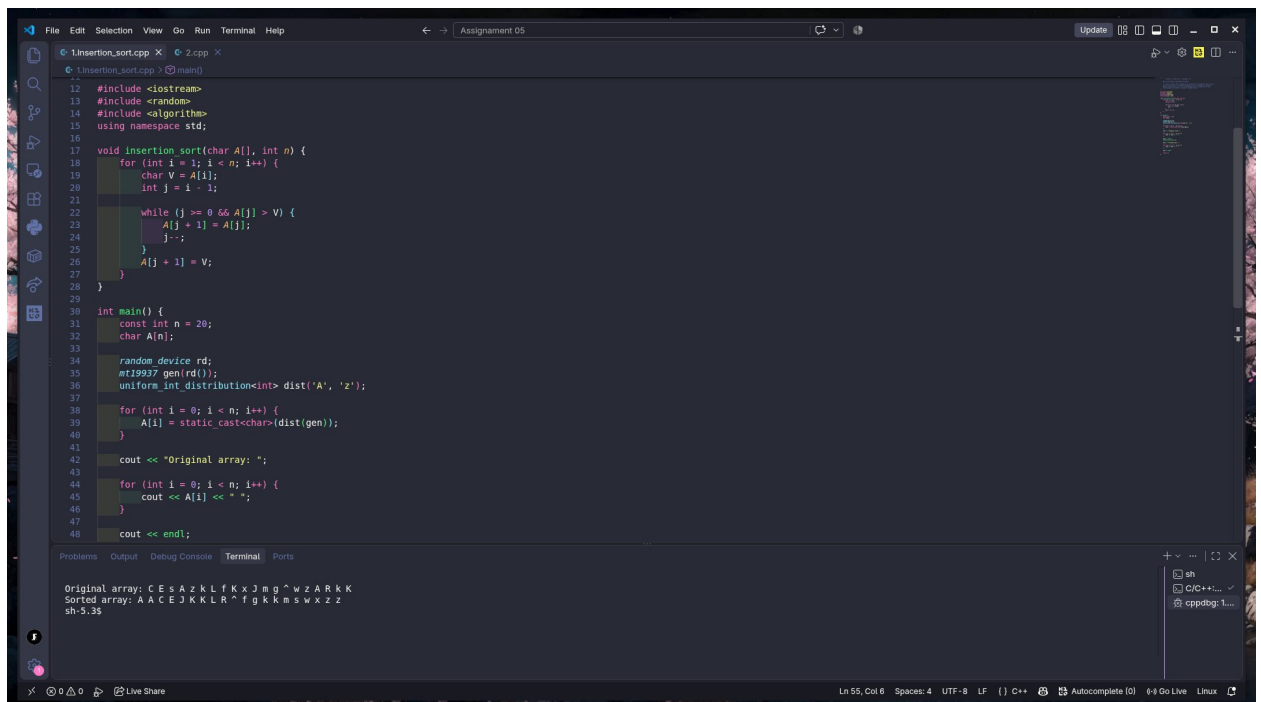
After: A B C F H

The best-case scenario occurs when the array is already sorted. Each element only needs to be compared with the preceding element, resulting in linear behavior.

$$\Theta(n)$$

The worst-case scenario occurs when the array is sorted in reverse order. Each new element may need to shift across the entire sorted portion of the array, resulting in quadratic behavior.

$$\Theta(n^2)$$



The screenshot shows a C++ program in a code editor. The program defines an `insertion_sort` function that takes a character array `A` and its size `n`. It uses a `while` loop to shift elements greater than the current element `V` one position to the right, then inserts `V` at the correct position. The `main` function generates a random array of 20 characters and prints the original and sorted arrays. The terminal output shows the original array as 'C E S A z k l f K x J m g ^ w z A R k K' and the sorted array as 'A A C E J K K L R ^ f g k k m s w x z z'.

```
12 #include <iostream>
13 #include <random>
14 #include <algorithm>
15 using namespace std;
16
17 void insertion_sort(char A[], int n) {
18     for (int i = 1; i < n; i++) {
19         char V = A[i];
20         int j = i - 1;
21
22         while (j >= 0 && A[j] > V) {
23             A[j + 1] = A[j];
24             j--;
25         }
26         A[j + 1] = V;
27     }
28 }
29
30 int main() {
31     const int n = 20;
32     char A[n];
33
34     random_device rd;
35     mt19937 gen(rd());
36     uniform_int_distribution<int> dist('A', 'Z');
37
38     for (int i = 0; i < n; i++) {
39         A[i] = static_cast<char>(dist(gen));
40     }
41
42     cout << "Original array: ";
43
44     for (int i = 0; i < n; i++) {
45         cout << A[i] << " ";
46     }
47
48     cout << endl;
49
50     insertion_sort(A, n);
51
52     cout << "Sorted array: ";
53     for (int i = 0; i < n; i++) {
54         cout << A[i] << " ";
55     }
56     cout << endl;
57 }
```

Original array: C E S A z k l f K x J m g ^ w z A R k K
Sorted array: A A C E J K K L R ^ f g k k m s w x z z
sh-3.3\$

2. Generate a list of 10000000 (ten million) random numbers with C++. Use the Binary Search and the Interpolation Search to find a key. Measure and report the execution time for each search.

Binary Search uses a **Decrease-by-a-Constant-Factor** strategy. At each iteration, the algorithm compares the search key with the middle element of the current range. Depending on the comparison:

- If the key equals the middle element, the search ends.
- If the key is smaller, the right half is discarded.
- If the key is larger, the left half is discarded.

Therefore, approximately half of the remaining elements are eliminated at each step.

Its recurrence can be represented as:

$$T(n)=T(n/2)+\Theta(1)$$

which gives:

$$T(n) = \Theta(\log n)$$

Interpolation Search uses a **Variable-size Decrease** strategy. It estimates the probable position of the key based on its value and the boundary values of the current search range. The estimated position is:

$$x = l + \left\lfloor \frac{(v - A[l])(r - l)}{A[r] - A[l]} \right\rfloor$$

Experimental Results

The array was generated with:

$n=10,000,000$

The searches were then executed and measured using C++ chrono.

First execution

The screenshot shows a C++ IDE with two source files: `1.insertion_sort.cpp` and `2.cpp`. The `2.cpp` file contains the following code:

```
19 int binary_search(const vector<int>& A, int key) {
20     int left = 0;
21     int right = A.size() - 1;
22
23     while (left <= right) {
24         int mid = left + (right - left) / 2;
25
26         if (A[mid] == key)
27             return mid;
28         if (A[mid] < key)
29             left = mid + 1;
30         else
31             right = mid - 1;
32     }
33     return -1;
34 }
35
36
37
38 int interpolation_search(const vector<int>& A, int key) {
39     int left = 0;
40     int right = A.size() - 1;
41
42     while (left <= right && key >= A[left] && key <= A[right]) {
43         if (left == right) {
44             if (A[left] == key)
45                 return left;
46             return -1;
47         }
48
49         long long pos = left + static_cast<long long>(key - A[left]) *
50             (right - left) / (A[right] - A[left]);
51
52         if (A[pos] == key)
53             return pos;
54     }
55     return -1;
56 }
```

The terminal output shows the results of the first execution:

```
Binary Search:
Position: 4999999
Execution time: 0 microseconds

Interpolation Search:
Position: 4999999
Execution time: 1 microseconds
sh-3.3$
```

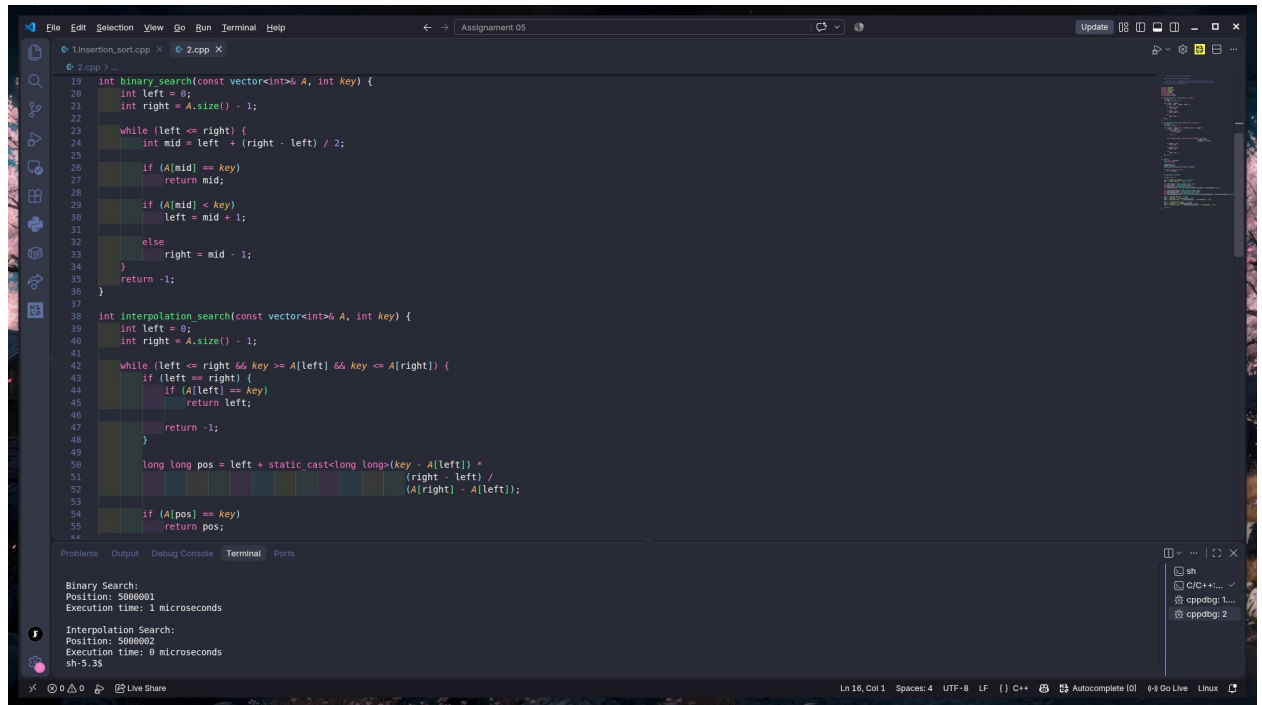
Binary search:

Position: 4999999 || Execution time: 0 microseconds.

Interpolation search:

Position: 4999999 || Execution time: 1 microseconds.

Second execution



```
19 int binary_search(const vector<int>& A, int key) {
20     int left = 0;
21     int right = A.size() - 1;
22
23     while (left <= right) {
24         int mid = left + (right - left) / 2;
25
26         if (A[mid] == key)
27             return mid;
28
29         if (A[mid] < key)
30             left = mid + 1;
31
32         else
33             right = mid - 1;
34     }
35     return -1;
36 }
37
38 int interpolation_search(const vector<int>& A, int key) {
39     int left = 0;
40     int right = A.size() - 1;
41
42     while (left <= right && key >= A[left] && key <= A[right]) {
43         if (left == right) {
44             if (A[left] == key)
45                 return left;
46             return -1;
47         }
48
49         long long pos = left + static_cast<long long>(key - A[left]) *
50             (right - left) /
51             (A[right] - A[left]);
52
53         if (A[pos] == key)
54             return pos;
55     }
56 }
```

Problems Output Debug Console Terminal Ports

Binary Search:
Position: 5000001
Execution time: 1 microseconds

Interpolation Search:
Position: 5000002
Execution time: 0 microseconds
sh-2.35

Binary search:

Position: 5000001 || Execution time: 1 microseconds.

Interpolation search:

Position: 5000002 || Execution time: 0 microseconds.

The different positions are expected because the array is randomly generated on each execution. Additionally, if duplicate values exist, the same key can occur at multiple positions.

The execution times of 0–1 microseconds should not be interpreted as a definitive performance difference. The individual search is so fast that the measurement is close to the resolution of the timer. Consequently, repeated searches can provide a more stable experimental measurement.

The theoretical complexity predicts:

Algorithm	Average Complexity	Worst Case
Binary Search	$O(\log n)$	$O(\log n)$
Interpolation Search	$O(\log \log n)$	$O(n)$

The experimental measurements are consistent with the fact that both algorithms perform the search extremely quickly, even for an array of 10 million elements.

However, the measured times alone are not sufficient to demonstrate the theoretical difference between $(O(\log n))$ and $(O(\log \log n))$, because the measured execution time is only 0–1 microseconds.

Therefore, the mathematical analysis provides the asymptotic behavior, while the experimental analysis measures the actual execution time on the computer.

3. Prove that the topological sorting problem has a solution if and only if it is a DAG.

Let $(G=(V,E))$ be a directed graph. A Directed Acyclic Graph (DAG) is a directed graph that does not contain directed cycles. A topological order is a linear arrangement of vertices such that, for each directed edge, vertex u appears before v . We must demonstrate:

$$G \text{ has to topological ordering} \iff G \text{ is a DAG}$$

1: If G is a DAG, then there exists a topological order

Let's assume that G is a DAG. Every DAG has at least one **source vertex**, that is, a vertex with an in-degree equal to zero. We select a source and place it at the beginning of the topological order. Then we remove that vertex and its outgoing edges. The remaining graph continues to be a DAG, because removing vertices and edges cannot create a cycle. Therefore, we can repeat the process until all vertices are eliminated.

$$v_1 \rightarrow v_2 \rightarrow v_3 \rightarrow \dots \rightarrow v_k$$

Since each vertex is selected when it has no incoming edges, all its edges point towards vertices that will appear later in the sorting.

Therefore, the resulting sequence is a topological order. So:

$$G \text{ is DAG} \Rightarrow G \text{ has order topological}$$

2: If G has a topological order, then G is a DAG

Let us now suppose that G has a topological order. Let us suppose, by contradiction, that G contains a directed cycle:

$$v_1 \rightarrow v_2 \rightarrow v_3 \rightarrow \dots \rightarrow v_k \rightarrow v_1$$

Since the edge $(v_1 \rightarrow v_2)$ exists, the topological order requires:

$$v_1 < v_2$$

The same way:

$$(v_2 < v_3)(v_3 < v_4)$$

and so on, until you obtain:

$$v_k < v_1$$

Therefore we would have:

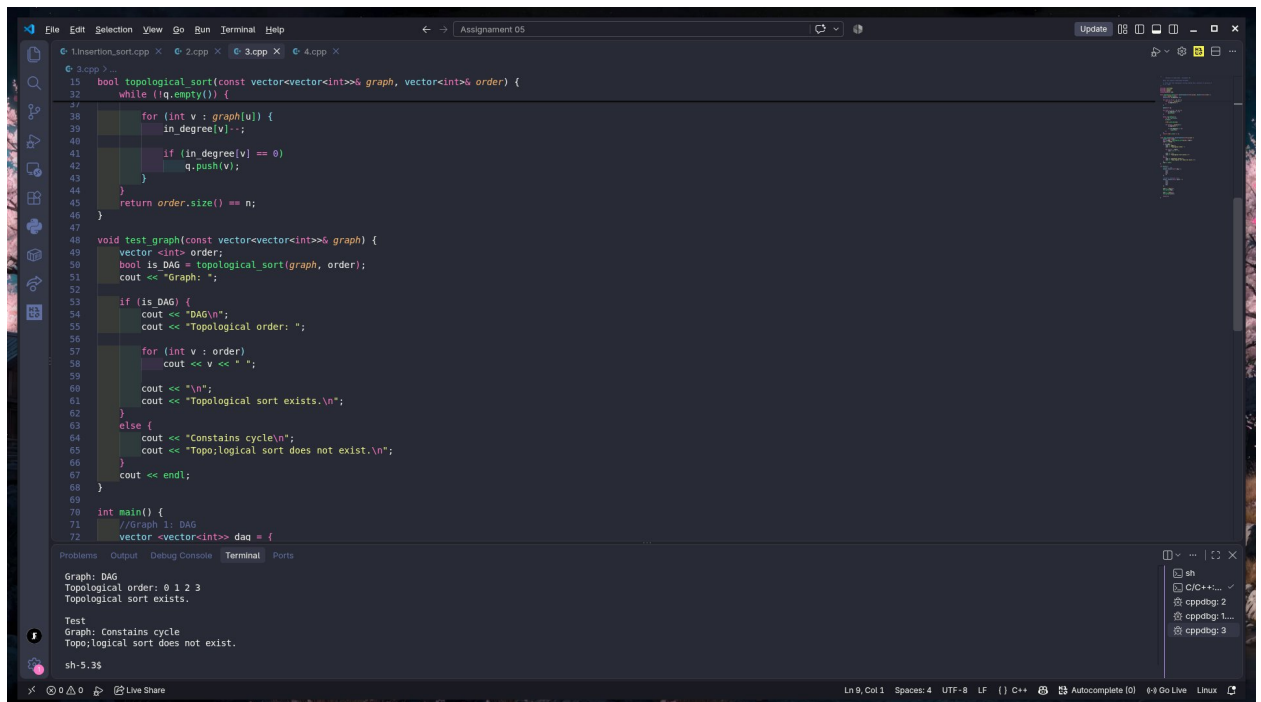
$$v_1 < v_2 < v_3 < \dots < v_k < v_1$$

This is a contradiction, because no element can appear before itself in a linear order.

Therefore, G cannot contain a directed cycle.

So:

G has topological order $\Rightarrow G$ is DAG



```
15 bool topological_sort(const vector<vector<int>>& graph, vector<int>& order) {
16     while (!q.empty()) {
17         for (int v : graph[u]) {
18             in_degree[v]--;
19             if (in_degree[v] == 0) {
20                 q.push(v);
21             }
22         }
23     }
24     return order.size() == n;
25 }
26
27 void test_graph(const vector<vector<int>>& graph) {
28     vector<int> order;
29     bool is DAG = topological_sort(graph, order);
30     cout << "Graph: ";
31
32     if (is DAG) {
33         cout << "DAG\n";
34         cout << "Topological order: ";
35         for (int v : order) {
36             cout << v << " ";
37         }
38         cout << "\n";
39         cout << "Topological sort exists.\n";
40     }
41     else {
42         cout << "Contains cycle\n";
43         cout << "Topological sort does not exist.\n";
44     }
45     cout << endl;
46 }
47
48 int main() {
49     // Graph 1: DAG
50     vector<vector<int>> dag = {
51         {1, 2, 3},
52         {2, 3},
53         {}
54     };
55     test_graph(dag);
56
57     // Graph 2: Cycle
58     vector<vector<int>> cycle = {
59         {1, 2},
60         {2, 1},
61         {}
62     };
63     test_graph(cycle);
64 }
```

Graph: DAG
Topological order: 0 1 2 3
Topological sort exists.

Test
Graph: Contains cycle
Topological sort does not exist.

4. For a DAG with n vertices, what is the largest number of distinct solutions the topological sorting problem can have? Show your solution.

Let $(G=(V,E))$ be a DAG with (n) vertices. We want to determine the maximum possible number of distinct topological arrangements.

Step 1: Maximum number of possible orderings

A set of n vertices can be ordered in all possible ways. The number of permutations of n elements is:

$$n !$$

For example, for $n = 3$:

$$n = 3 : 3!=3\times 2\times 1 = 6$$

Therefore, no graph with n vertices can have more than $n!$ topological arrangements.

Step 2: Building a DAG that reaches $n!$

Let's consider a graph with n vertices and no edges :

$$E = \emptyset$$

This graph is a DAG because it does not contain any directed cycles. Furthermore, since there are no edges, there are no precedence restrictions between vertices. Therefore, any permutation of the n vertices constitutes a valid topological order. The number of solutions is then:

$$n!$$

Step 3: Prove that it is the maximum

Suppose we add an edge:

$$u \rightarrow v$$

Now there is a restriction: u must appear before v . Therefore, some of the permutations are no longer valid. Each additional edge can impose additional restrictions, but cannot increase the number of topological arrangements. Consequently, the maximum number is obtained when there are no restrictions. Therefore:

Maximum number of topological arrangements = $n!$

Example

For $n = 5$, we consider:

$$V = \{1, 2, 3, 4, 5\}$$

and:

$$E = \emptyset$$

The maximum number of topological arrangements is:

$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$$

Therefore, this DAG has 120 distinct topological solutions.

```

22
23
24 int main() {
25     int n = 5;
26     vector<int> vertices(n);
27     for (int i = 0; i < n; i++)
28         vertices[i] = i + 1;
29
30     long long count = 0;
31
32     do {
33         count++;
34         cout << "Ordering " << count << ": ";
35         for (int v: vertices)
36             cout << v << " ";
37         cout << endl;
38     } while (next_permutation(vertices.begin(), vertices.end()));
39
40     cout << "Total number of distinct solutions: " << count << endl;
41     cout << "Permutation of " << n << " vertices: " << permutation(n) << endl;
42     return 0;
43 }
44
45
46
47
48

```

Ordering 105: 4 1 2 0 3
Ordering 106: 4 1 2 3 0
Ordering 107: 4 1 3 0 2
Ordering 108: 4 1 3 2 0
Ordering 109: 4 2 0 1 3
Ordering 110: 4 2 0 3 1
Ordering 111: 4 2 1 0 3
Ordering 112: 4 2 1 3 0
Ordering 113: 4 2 3 0 1
Ordering 114: 4 2 3 1 0
Ordering 115: 4 3 0 1 2
Ordering 116: 4 3 0 2 1
Ordering 117: 4 3 1 0 2
Ordering 118: 4 3 1 2 0
Ordering 119: 4 3 2 0 1
Ordering 120: 4 3 2 1 0
Total number of distinct solutions: 120
Permutation of 5 vertices: 120
sh-5.3\$